

MCP SERVER LAB

Building an MCP Server on Top of Your Company's Data

THE TASK

You'll be placed in a new team of three, in a new shared GitHub repo, on a new company and industry. Your job is to invent a single realistic problem that company would actually face where the fix is *giving an LLM safe, scoped access to company data*, and then build that access the right way: a real database with an ERD, and an MCP server sitting in front of it that a model talks to instead of the database directly.

You're building a server that handles a set of protocol-level behaviors we covered in the MCP material beyond just "the model calls a tool and gets an answer" because these are exactly the mechanisms that let a server stay safe and predictable once real users, real role changes, and real mistakes show up.

THE PROTOCOL CONCERNS

We covered a full data layer and transport layer, and your server needs to demonstrate all of it. **Your team must implement every concern below.** Design your problem so that each one has a genuine reason to exist in your system a forced elicitation on a harmless tool is worse than not having one, so pick a problem with enough real risk and real state change to justify all of them honestly, Actually research the company and find something new, a "customer chatbot" is not acceptable.

- **Capability negotiation:** a real initialize / initialized exchange: your server declares which capabilities it actually supports, and your client checks those declarations before relying on them, rather than assuming the other side supports everything.
- **Notifications:** the tool set is not static for the life of a connection. Design a situation where it genuinely changes at runtime (a role changes, a state changes, a resource becomes available) and push tools/list_changed instead of making the client poll or guess, and show how that changes the ongoing conversations with your agent.
- **Elicitation:** a tool that, mid-call, needs to stop and ask a human for more information or explicit confirmation before it can safely finish actually using elicitation/create, not silently proceeding or silently failing. (You should research this)
- **Resources:** something in your domain is better modeled as data the model can *fetch* than a function it *calls*, a policy document, a read-only record, a report exposed via resources/list and resources/read rather than wrapped in a tool.
- **Prompts:** a reusable prompt template your server exposes so the host can surface a canned, parameterized starting point for a common task (e.g., "draft a refund explanation for order {order_id}") instead of every client re-inventing that prompt.
- **Transport (Both):** don't just default to stdio because it's the example in the material decide. Your server should be local (stdio) while in development then transition to remote (Streamable HTTP) as you finish your project.. I want to see these changes happen in the commit history as you work.
- **Progress tracking:** a tool whose work genuinely takes a while (a report generation, a batch lookup) that reports progress rather than leaving the client blocked with no feedback.
- **Defensive tool design:** take the Tool Specifications material seriously on at least one write tool, real JSON Schema constraints (not just types), server-side validation independent of the schema, and an authorization check that happens in the handler, not just "the schema says the ID is an integer so it must be fine."

If your project only implements concerns that are trivially safe, always-on, and never change, you haven't found the right problem yet, go back and pick one with real stakes and real state changes.

A WORKED EXAMPLE READ THIS, THEN DO SOMETHING DIFFERENT

This shows the shape of the exercise, not the problem to copy. Pick something genuinely different for your own company.

Picture "**Larkspur Veterinary Clinics**," a chain of small-animal vet clinics (not on your list). Staff want an assistant that can look up a pet's record, check appointments, and process prescription refills without raw access to the clinic database.

Database/ERD: clients, pets, appointments, vets, prescriptions, invoices where pets belong to clients; appointments and prescriptions reference both pets and vets.

Negotiation: the server declares elicitation support in initialize. The client checks that before offering the refill tool, a client without elicitation support gets a read-only fallback instead.

Notifications: front-desk sessions get read-only tools only. When a vet authenticates, the server pushes tools/list_changed and approve_refill / mark_appointment_complete appear, no reconnect needed.

Elicitation: approve_refill triggers elicitation/create mid-call when the medication is a controlled substance, pausing for a vet's explicit sign-off. An uncontrolled medication completes immediately.

Resources: the controlled-substance policy is a static document exposed via `resources/read` instead of a tool, so the model reads it once and reasons over it.

Transport: a single clinic is a stdio case. A multi-location chain pushes real deployment toward Streamable HTTP behind auth, the demo may still run over stdio for simplicity, but the README says which one you actually built and why.

Each concern here earns its place: the sign-off requirement is why elicitation matters, the role split is why notifications matter, the policy doc is why a resource beats a tool. This is about what the *server* is willing to do, not a bigger prompt.

WHAT TO BUILD AND SUBMIT

- A **README** describing your company in the light of the new problem (modify the old readme slightly), the problem your team invented, the ERD, how each protocol concern shows up in your problem.
- A ``db/`` folder: your schema (whatever engine you chose SQLite, Postgres, or otherwise), seed data, and the ERD (image or Mermaid source).
- An ``mcp_server/`` folder: your server code, with the input schema for every tool easy to find (typed fields, required, additionalProperties: false, real descriptions not placeholders).
- Inside `mcp_server/`, every concern should be easy for a grader to locate without reading the whole file. The capability declaration in `initialize`, the code path that fires a notification and what triggers it, the tool that calls elicitation/create and its trigger condition, the sampling call, the `resources/prompts` exposed, and so on.
- An ``agent/`` (or `client/`) folder: your agent wired to the server, showing the handshake actually happening, not assumed.
- A short **demo transcript or recording** (see below) showing every concern actually firing not just described in the README.
- A short **comparison note** in the README: which tools are read-only vs. write, which require elicitation and why, and what your server does if a client connects without a capability one of your riskier tools depends on.
- Commit history should show real contributions from all three of you split the database, the server, and the agent/demo work so no one owns more than two of the three pieces, and everyone can speak to the whole system.

THE PRESENTATION

Each team presents, and same as last time the presentation is a story, not a slide dump of code. Put yourselves inside the company: you three work there, this data-access problem landed on your desk, and you're telling your boss how you solved it. Present it in first person and in order, roughly like this:

- **The setup:** who you are at the company, and what people did before anyone thought to connect an LLM to this data. Make the risk feel real what could go wrong with the naive version, and why it mattered enough to design around.
- **Building the server:** how you scoped your tools, and one decision you made about a schema or a boundary check that you're glad you made.
- **Walk through each concern, live:** for every protocol concern, show it actually firing then the notification changing the tool set, the elicitation pausing for a human, the resource being read instead of called, the sampling call, the progress updates.
- **Where it stands now:** what you'd still worry about if this ran in production and what you'd want to try next.

Bring your comparison note as evidence inside the story, not as a separate appendix.

RESOURCES

- **Model Context Protocol specification** (modelcontextprotocol.io/docs/getting-started/intro and the [full spec](#)): the primary source for exact message shapes for `initialize`, `notifications/tools/list_changed`, `elicitation/create`, `sampling/createMessage`, and the `resources/prompts` primitives. Read the actual spec pages, not just a blog post about them.
- ``langchain-mcp-adapters`` ([GitHub](#), [PyPI](#)): if you're wiring your server into a LangChain agent, this handles discovery and dispatch for you.
- ``jsonschema`` ([PyPI](#), [docs](#)): for server-side validation of tool arguments, independent of whatever the model actually sends.
- [dbdiagram.io](#) ([dbdiagram.io](#)) and [drawSQL](#) ([drawsql.app](#)): free tools for building and exporting your ERD.
- **GitHub basics** ([GitHub Docs: Get started](#)): start here if anyone on your team is new to Git/GitHub: repositories, commits, branches, and working with a team on a shared repo.
- **Creating your shared repo** ([creating a new repository](#), [adding files to a repository](#)): the exact steps for one teammate to create the team's shared repo, add the other two as collaborators, and get everyone's first commit in.
- **GitHub Skills** ([skills.github.com](#)): free, hands-on courses like "Introduction to GitHub" and "Resolving merge conflicts" if your team wants practice before the lab starts.
- **Pro Git** (git-scm.com/book): the free, canonical reference for branching, merging, and resolving conflicts once all three of you are pushing to the same repo.
- **Reviewing and protecting the repo** ([collaborating with pull requests](#), [github/gitignore](#) templates): use pull requests so a teammate reviews your work before it merges, and start from a real `.gitignore` so an `.env` file or database credential never gets committed by accident.

A FEW GUARDRAILS

- Pick a problem where at least one tool is *genuinely* risky or state-dependent enough to justify the human/model-in-the-loop concerns. If every tool is safe and static, you haven't found the right problem yet.
 - Every concern needs a real reason to exist in your system, a concern bolted on with no genuine trigger will be obvious in the demo.
 - Every tool's input schema needs required fields and additionalProperties: false don't put bare dict or untyped **kwargs tools.
 - Never commit an API key or a database credential, to any provider. Use a .env file and add it to .gitignore.
 - Your server should behave reasonably if a client connects *without* a capability one of your tools needs. Don't assume every client supports elicitation or sampling.
 - Keep a small, fixed set of test inputs including at least one that should trigger each concern, so your demo is repeatable, not lucky.
-

WORKING AS A TEAM: ISSUES & MODULAR WORK

Teamwork inside the repo is graded directly, not assumed from the fact that three names appear in the commit log. Every **GitHub Issue** your team opens is graded on its *rationale*. why that unit of work needs to exist, what it depends on, and what "done" looks like not just on the fact that an issue exists. An issue that only says "do the refill tool" earns nothing; an issue that states the problem, the constraint, and the acceptance criteria does.

Breaking the project into modular parts: before opening any issues, split the work into pieces that map cleanly onto db/, mcp_server/, and agent/, and then again onto each of the eight protocol concerns (capability negotiation, notifications, elicitation, sampling, resources, prompts, transport, progress tracking, defensive tool design). Each concern should be its own issue with a single clear owner, so the repo shows who is responsible for which piece before any code is written, not just after the fact.

Writing an issue with real rationale: state the problem, not the task, e.g. "approve_refill has no human-in-the-loop check for controlled substances" rather than "add elicitation." Name the constraint that makes the concern genuinely necessary in your invented problem, and write acceptance criteria a teammate could check against without asking you what you meant.

Assigning, linking, and closing issues: assign every issue to exactly one owner, reference it from the pull request that resolves it (e.g. "Closes #12") so the resolution is traceable from issue to diff, and leave a short comment when you close it explaining what changed. An issue opened and closed within the same minute, with no linked commit or discussion, will not count as attributable teamwork.

Resources for working as a team:

- **About GitHub Issues** (docs.github.com): what an issue is for and how your team should use one per unit of work.
- **Creating an issue** ([creating an issue](#)) and **linking a pull request to an issue** ([linking a PR to an issue](#)): the exact mechanics for opening an issue and closing it through the PR that resolves it.
- **GitHub Projects** ([about Projects](#)): a free kanban-style board to track your modular pieces (to do / in progress / done) across all three of you.
- **Writing good issues** ([necolas/issue-guidelines](#)): a widely used community guide for what a well-reasoned issue actually contains.
- **GitHub flow** (docs.github.com): a lightweight branch-per-issue workflow that keeps three people from stepping on each other in the same repo.

GRADING RUBRIC

Your team deliverable is scored out of 100 fixed points, condensed below from the grading manifesto plus the added teamwork category above. Every row is binary where a category earns its full points only when its criteria are genuinely met, not partially. Your final individual grade is 70% this team score plus 30% an individual contribution score built from Git history, so commit history and issue rationale that actually show your own work matter as much as the deliverable itself.

Category	Points	What's assessed
Problem framing & suitability	10	Company/problem fit, originality vs. the worked example, and whether the invented problem genuinely needs all eight protocol concerns.
Database & ERD	10	Schema supports every tool, ERD matches the schema, seed data covers normal and edge cases, engine is documented.
MCP server & tool specs	15	No raw SQL/shell tools, typed and constrained schemas, required/additionalProperties: false, real descriptions, server-side validation, handler-level authorization.
Capability negotiation	5	A real initialize/initialized exchange, specific declared capabilities, and a client that checks them before relying on a capability.
Notifications	7	A genuine runtime change to the tool set, a real tools/list_changed push, and a client that reacts to it.
Elicitation	5	A genuine trigger condition, a real elicitation/create call that pauses for a response, and an outcome gated on that response.
Sampling	8	A genuine need for reasoning, a real sampling/createMessage call through the client's model (not the server's own), and the result actually used.

Category	Points	What's assessed
Resources & prompts	5	At least one true resource (fetched, not called) with correct content, and a discoverable, parameterized prompt template.
Transport choice, justified	5	The transport actually built (stdio vs. Streamable HTTP) stated in the README, with a justification tied to the specific problem.
Progress tracking	5	A genuinely long-running tool that reports real intermediate progress, not just a single blocking response.
Agent/client integration	10	The agent genuinely discovers and calls tools on the server, an end-to-end demo path, and complete run instructions.
Repository usability & safety	5	Clear organization, demo evidence covering every concern, reproducible configuration, and no exposed secrets.
Teamwork & issue rationale	10	GitHub Issues opened for each modular unit of work, each with a genuine rationale (problem, constraint, acceptance criteria) rather than a bare task label, assigned to a single owner and closed through a linked, traceable pull request.
Total	100	